



Network intrusion response systems: towards standardized evaluation of intrusion response

Thomas Marchioro, Rachida Saroui, Alexis Olivereau

► To cite this version:

Thomas Marchioro, Rachida Saroui, Alexis Olivereau. Network intrusion response systems: towards standardized evaluation of intrusion response. ESORICS 2025: 30th European Symposium on Research in computer security, Sep 2025, Toulouse, France. <hal-05294762>

HAL Id: hal-05294762

<https://hal.science/hal-05294762v1>

Submitted on 3 Oct 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Network Intrusion Response Systems: Towards standardized evaluation of intrusion response

Thomas Marchioro^[0000–0003–3353–102X], Rachida Saroui, and Alexis Olivereau

Université Paris-Saclay, CEA, List, F-91120, Palaiseau, France
`thomas.marchioro@cea.fr,rachida.saroui@cea.fr,alexis.olivereau@cea.fr`

Abstract. While intrusion detection has become increasingly relevant and standardized, the same cannot be said for intrusion response. Currently, intrusion response systems vary widely in terms of infrastructure requirements, evaluation methodologies, and performance metrics. In this paper, we take a step toward standardizing intrusion response by introducing the concept of *network intrusion response systems* (NIRS). NIRS rely solely on network traffic data and perform network-level actions, thereby requiring minimal assumptions about the underlying infrastructure. We motivate this new definition by: (a) introducing the fundamental problem that NIRS aim to solve, i.e., turning alerts raised by network intrusion detection systems (NIDS) into network-level actions; (b) illustrating desirable properties that are fulfilled and their devised workflow in a network ecosystem. Finally, we discuss how NIRS, differently from other response solutions, can be evaluated on publicly available datasets that are typically used for NIDS assessment, thereby facilitating performance comparison between different implementations. We demonstrate how this evaluation can be performed on two baseline NIRS approaches: a heuristic-based and an LLM-based NIRS.

Keywords: intrusion response · evaluation protocols · network security.

1 Introduction

Network intrusion detection systems (NIDS) have been steadily gaining traction in both the scientific community and industry, with a multitude of new solutions, methodologies, and benchmarks published each year [2]. There are several key factors contributing to the success of NIDS. Firstly, NIDS are portable solutions. They can be deployed on virtually any network with minimal requirements: all they need is the permission to inspect the network traffic. This allows to apply them to a wide range of ecosystems. A second key factor is the availability of multiple public datasets for evaluation. NIDS datasets, albeit subject to several limitations, enable researchers to easily validate their solutions, compare them with the state of the art, and identify biases or methodological flaws in existing work. A third reason for the steady progress of NIDS is the adoption of a well-established set of metrics and evaluation protocols. It is well understood that metrics such as true positive rate (TPR) and the false positive rate (FPR), as

well as the execution time, are fundamental to assess NIDS performance. These metrics are computed using evaluation protocols designed to simulate realistic deployment scenarios—for example, employing temporal data splitting rather than random splitting or assessing robustness to adversarial inputs.

In contrast to the growing popularity of NIDS, intrusion response systems (IRS) have lagged behind in both success and accumulated knowledge. This is largely due to a lack of consensus within the scientific community and industry regarding the role and scope of these systems. The definition of IRS itself varies significantly across different studies, as reflected in literature reviews that struggle to establish a consistent taxonomy [8,5]. At its core, an IRS is any system that takes action in response to an intrusion; however, this broad definition means that virtually any action performed post-detection can be considered “intrusion response.” The lack of a unified definition also results in IRS being evaluated in different ways (see §2.2 for details). Assessing IRS requires to measure their impact on the network, both in terms of mitigating attacks and preserving normal network behavior. In many cases, this cannot be trivially done using static datasets. Therefore, research work typically evaluate IRS via simulators or virtualized testbeds, which vary a lot in assumptions and complexity, and are often not publicly released. Performance metrics and evaluation protocols also vary significantly across studies. This heterogeneity in scope, metrics, and evaluation methodology makes it difficult, if not impossible, to directly compare different IRS solutions, ultimately hindering progress and preventing the development of a shared body of knowledge for intrusion response.

In this paper, we define a specific type of IRS, called network intrusion response systems (NIRS), which have a specific objective and scope of action, and can be evaluated on public NIDS datasets. This new definition is motivated by a practical problem: converting NIDS alerts into network-level actions (§3.1). NIRS aim to tackle this problem by ingesting NIDS alerts and producing rules that can be straightforwardly applied to the monitored network. After detailing specifications, properties, and workflow (§3.3) of these systems, we illustrate how they can be evaluated on public NIDS datasets (§3.4). Finally, we apply our proposed evaluation methodology to two baseline NIRS (§4).

Contributions The contributions of this paper are as follows:

- We introduce a specific category of IRS, called network intrusion response systems (NIRS), that provide actionable network-level responses and can be evaluated on public NIDS datasets.
- We illustrate properties and our devised workflow for NIRS, providing examples of how they can be integrated in a network ecosystem.
- We detail an evaluation methodology and a set of metrics that can be employed to compare NIRS performance on public NIDS datasets.
- We provide two examples of NIRS and apply our proposed evaluation methodology using a public NIDS dataset, namely NB15.

Reproducibility and artifact disclosure. To allow the reproducibility of the results presented in this paper and favor future research on NIRS, all the code used in

the paper has been made publicly available on Github at the following link: <https://github.com/thomasmarchioro3/NIRS>.

2 Background and Related Work

In this section, we provide relevant background on network intrusion detection and briefly review how some key aspects of intrusion response are addressed in the existing literature. We then highlight the need for standardization in cybersecurity and explain how this paper aligns with standardization efforts in other domains.

2.1 Network intrusion detection

Network intrusion detection systems (NIDS) are the most prominent type of intrusion detection systems. NIDS monitor network traffic, either in the form of packets or NetFlows, and raise alerts when the traffic is attributed to malicious activity. In the industry, the most adopted NIDS are signature-based, like Snort and Suricata¹. However, because signature-based NIDS are vulnerable to evasion techniques and zero-day attacks, machine learning (ML)-based NIDS have been increasingly adopted alongside them [13]. ML-based NIDS are essentially data-driven classifiers that map network traffic to a binary label (*benign* or *malicious*).

Most ML-based NIDS are trained and evaluated on labeled datasets of NetFlows such as CIC-IDS 2017 or TON-IoT. A NetFlow x is a tuple that characterizes a specific connection between two hosts, e.g.,

$$x = (t, \text{src_ip}, \text{src_port}, \text{dst_ip}, \text{dst_port}, \text{protocol}, \dots) \quad (1)$$

where t is the first packet's timestamp for the NetFlow, **src_ip** (**src_port**) and **dst_ip** (**dst_port**) are the the source and destination IP addresses (ports), and **protocol** is the network protocol used by the two hosts to communicate. Other information included in a Netflow can vary depending on the tool used for monitoring the network traffic (e.g., CICFlowMeter, Zeek, NFStream²).

2.2 Intrusion Response

As mentioned in the introduction, IRS vary significantly on several aspects. We emphasize three of them, namely the type of response action, evaluation methodologies, and metrics, that our proposed notion of NIRS aims to address.

¹ <https://www.snort.org/>, <https://suricata.io/>

² <https://github.com/UNBCIC/CICFlowMeter>, <https://zeek.org/>,
<https://www.nfstream.org/>

Response actions Response actions can be categorized into three main groups based on the level of control they exercise on the network. Firewall-based responses interact with existing routers or firewalls [22] to block or redirect traffic. SDN-based responses rely on the presence of a software defined networking (SDN) infrastructure, allowing fine-grained and dynamic control over NetFlows, e.g., by dynamically rerouting traffic or applying rate limits [20,21]. While these approaches provide powerful control mechanisms, they are limited in applicability, as few real-world networks are endowed with an SDN infrastructure. Host-level responses assume the ability to intervene directly on hosts, which may involve actions like patch deployment, service restart, process isolation, or migration [11,28]. Such approaches offer rich recovery options but require privileged access and strong orchestration capabilities.

Evaluation methodologies The way IRS are evaluated typically falls under three categories. Theoretical analysis relies on system abstraction and assumptions to reason about the IRS performance [15,26]. Simulated environments [28] can be used to model essential aspects of a system. However, most simulators are not publicly disclosed, limiting reproducibility, with notable exceptions such as MiniCPS [1] and CyberBattleSim [25]. Virtualized testbeds can be a valuable alternative to simulators, creating a more realistic scenario using virtual machines [22], containers, or network emulation platforms (e.g., GNS3 as in [11]). However, they also are rarely released publicly and their scope is usually limited to small networks due to hardware limitations.

Metrics Evaluation metrics also vary widely across the literature, with no consensus on standardized indicators. Attack-related indicators such as the percentage of blocked attacks [22] or the final stage reached in a multi-stage attack scenario [11] are commonly used. Response time is also adopted to assess how quickly the system reacts to a threat [12]. QoS-related indicators can quantify the impact of the response on legitimate traffic; examples include preserved throughput, increased latency, or false positive disruption [20]. Physical-level indicators, typically applicable in cyber-physical or industrial settings, reflect the safety of physical processes (e.g., likelihood of tank overflows [21] or power load issues [15]). Reward or utility functions, mostly used in RL-based and game-theoretic IRS, are used to guide the decision-making process, but they are also employed as evaluation metrics. These functions typically integrate multiple heterogeneous objectives and tend to be system-specific [20,28].

2.3 Standardization efforts in ML-based cybersecurity

Standardization is a critical concern in machine learning research, especially in ML-based cybersecurity. Designing data-driven security solutions involves subtle challenges, and without standardized protocols, it is easy to fall into methodological pitfalls. Common issues include neglecting the problem domain or overlooking biases that can significantly distort evaluation results [19]. Over the years, consistent efforts were made to establish methodologies for evaluating

ML-based security solutions [4]. This is true for NIDS as well. A primary issue in NIDS is dataset errors and biases, which stem from poor representation of benign traffic or attacks. For the most widespread datasets, however, these biases are now largely acknowledged [14]. Evaluation biases are also common. For example, many past works used to adopt random data splitting for evaluation, which ignores temporal dependencies. At present, temporal splitting and concept drift analysis have become standard requirements in NIDS evaluation [2]. Transferability is also major concern. Although NIDS are very portable in terms of deployability, it can be hard to transfer their knowledge between different networks [3]. This can be attributed to a number of reasons, one being the lack of standardized NIDS features [6], which is an open problem to this day.

3 Network Intrusion Response Systems

In this section, we introduce the problem that network intrusion response systems (NIRS) aim to solve (§3.1), we provide a formal definition for them (3.2), and list a number of desirable properties that these systems meet (§3.3). Then, we illustrate a NIRS workflow in practical deployments, and show how NIRS can be evaluated (§3.4).

3.1 The network intrusion response problem

As discussed in §2.1, a NIDS can be seen as a function $\mathcal{D} : \mathbb{X} \rightarrow \{0, 1\}$ that maps connections $x \in \mathbb{X}$ (where $\mathbb{X}$ is a space of possible packets or NetFlows) to predicted labels $\hat{y} = \mathcal{D}(x) \in \{0, 1\}$, with $\hat{y} = 0$ denoting normal traffic and $\hat{y} = 1$ denoting an alert. A sequence of NIDS outputs is as shown in Table 1.

t	src_ip	src_port	dst_ip	dst_port	protocol	$\hat{y}$
1556172725	192.168.1.30	42966	52.59.177.21	80	tcp	1
1556173461	192.168.1.31	34532	18.184.104.180	80	tcp	0
1556340869	192.168.1.32	50204	18.194.169.124	80	tcp	0
1556340895	192.168.1.32	45016	176.28.50.165	80	tcp	1
1556548974	192.168.1.34	11	74.125.109.8	-	icmp	1
...	...	...	...	...	...	...

Table 1. Example of a NIDS output.

A continuous stream of alerts can be difficult to manage, especially when monitoring networks that handle large volumes of traffic. Additionally, among these alerts there is often a consistent portion of false positives. This is inevitable, since, when the traffic reaches the order of GB/s or TB/s, even a very low false-positive rate can result in a large volume of alerts.

While some forms of aggregation or meta-rules can be used to merge alerts, these will not be useful unless they can help identify an effective response. In other words, to be effective, the aggregation should be action-oriented: the system should not simply try to group similar alerts and identify false positives; it should also provide insights on how to contrast incoming malicious traffic. For

this reason, in this paper we focus on the problem of mapping NIDS alerts to practical network-level actions. We call this problem *network intrusion response*.

3.2 NIRS: Definition

To solve the network intrusion response problem, we define network intrusion response systems (NIRS) as systems that dynamically update a set of network rules (henceforth, ruleset) for a monitored network. The ruleset determines which incoming packets are accepted in the network. In practice this can be applied in the form of firewall rules, such as `iptables`. The ruleset is updated by the NIRS based on a set of labeled connections. These connections should include not only alerts, but also examples of regular traffic, so as to gauge the impact that a rule update may have on it. Formally, we can define a network ruleset as a map $r : \mathbb{X} \rightarrow \{0, 1\}$ that takes as input a connection $x \in \mathbb{X}$ (with $\mathbb{X}$ being the space of possible packets or NetFlows) and outputs a decision $b = r(x) \in \{0, 1\}$, which can be either to accept (0) or block (1) the connection. Labeled connections can be defined as tuples

$$\bar{x} = (t, \text{src_ip}, \text{src_port}, \text{dst_ip}, \text{dst_port}, \text{protocol}, \dots, \hat{y}) \quad (2)$$

i.e., adding the NIDS prediction $\hat{y}$ and other NIDS metadata to a connection (where a connection is either a single packet or Netflow). This leads to the following formalization of the NIRS definition.

Definition 1. *A network intrusion response system (NIRS) is a function $\mathcal{R}$ that maps*

$$(r_t, \bar{X}_A, \bar{X}_N) \mapsto r_{t+\Delta t} = \mathcal{R}(r_t, \bar{X}_A, \bar{X}_N) \quad (3)$$

with input:

- r_t : The current network ruleset.
- $\bar{X}_A$: A set of network connections $\{\bar{x}_{A,1}, \bar{x}_{A,2}, \dots\}$ that were labeled as alerts by a network intrusion detection system (NIDS).
- $\bar{X}_N$: A set of network connections $\{\bar{x}_{N,1}, \bar{x}_{N,2}, \dots\}$ that were labeled as normal traffic by a NIDS.

and output:

- $r_{t+\Delta t}$: An updated network ruleset. $\Delta t > 0$ is the time elapsed between the enforcement of two rulesets.

3.3 NIRS: Properties and workflow

The advantages of adopting NIRS, as opposed to a more general notion of IRS, can be summarized through the following key properties:

- P1. *Portability.*** NIRS can be deployed on any system that meets minimal requirements, thanks to their reliance on network-level rule updates. Since they operate by modifying rules, such as iptables or similar mechanisms, NIRS can be integrated into any network environment that supports rule enforcement. This includes basic firewalls or Intrusion Prevention Systems (IPS) placed at network entry points.
- P2. *Actionability.*** NIRS translate alerts $\bar{X}_A$ and normal traffic $\bar{X}_N$ into executable rulesets that can be immediately applied as a real-time response. This makes them more actionable compared to solutions that focus solely on refining alert classification or providing additional context.
- P3. *Comparability.*** Since NIRS operate with well-defined input and output structures, two NIRS can be directly compared under a fixed experimental setup. This is in contrast to other IRS implementations that rely on diverse and often incompatible assumptions.
- P4. *Measurability.*** The output of a NIRS (i.e., the ruleset r_t) functions effectively as a binary classifier, determining whether to accept (0) or block (1) each individual connection. This characteristic naturally supports the definition of standard evaluation metrics based on classification outcomes, such as correctly and wrongly blocked connections, as elucidated in §3.4.

While the formal definition is meant to be general and flexible, we also provide a general workflow for the practical deployment of NIRS, which is shown in fig. 1. In this workflow, network traffic between two hosts—one internal and one external to the monitored network—passes through a firewall/IPS with ruleset r_t . Any connection x that is not dropped by the firewall/IPS is monitored by the NIDS $\mathcal{D}$, which labels it as an alert or normal traffic. The labeled connection $\bar{x}$ is added to a data store. Alerts and normal connections are queried from the data store and processed by the NIRS $\mathcal{R}$, which proposes a ruleset update $r_{t+\Delta t}$. The proposed update is then evaluated by a security analyst, who finally decides whether to update the current ruleset. Clearly, when evaluating $\mathcal{R}$, the human in the loop is ignored and the proposed updates are always assumed to be accepted. While the use of response systems to dynamically update firewalls is not new [22], framing these solutions using a formal framework helps facilitating their comparison.

3.4 NIRS: Evaluation

Evaluation setup Like other types of intrusion response systems, NIRS can be evaluated using interactive testbeds. However, a key advantage of NIRS is the possibility of evaluating them on public NIDS datasets.

A NIDS dataset $\mathbb{D} = \{(x_i, y_i)\}_{i=1}^n$ contains pairs of NetFlows x_i and their ground-truth labels y_i . A NIRS can be evaluated on these datasets by iterating over labeled NetFlows (x_i, y_i) and comparing their ground-truth label with the decision b_i of the ruleset r_t in place at timestamp t_i . An important aspect to be considered in NIRS evaluation is that a ruleset based on a certain NIDS alert is put in place only after the flow responsible for that alert has ended. In

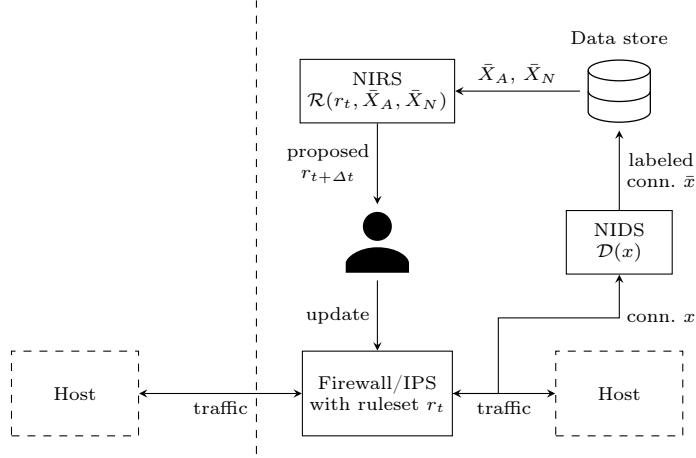


Fig. 1. Network intrusion response workflow.

other words, NIRS cannot retroactively block alerts. Clearly, NetFlows should be processed in order of their timestamps to emulate a real-time scenario. Furthermore, the specifics of the NIRS implementation should also be considered in the evaluation. For example, for a NIRS $\mathcal{R}$ that updates r_t at a constant rate Δt , the NetFlows can be evaluated in batches covering time ranges of size Δt , as described by Algorithm 1. For each NetFlow x_i with timestamp $t_i \in [t, t + \Delta t]$, the algorithm first checks whether it is blocked by the current ruleset r_t (i.e., if $b_i = r_t(x_i)$ is 1). After verifying whether x_i is blocked, one of the counters (CB, WB, CA, WA) is updated depending on the values of b_i and the ground truth y_i . If x_i is blocked, no further step is needed. Otherwise, x_i is given as input to the NIDS $\mathcal{D}$ to obtain a prediction $\hat{y}_i = \mathcal{D}(x_i)$. x_i is then added to $\bar{X}_A$ or $\bar{X}_N$ depending on the value of $\hat{y}_i$. When all the NetFlows in the current time range have been processed, the NIRS $\mathcal{R}$ is invoked to update the ruleset. It should be noted that this algorithm does not account for the ruleset-installation time—i.e., the time τ employed by $\mathcal{R}$ to generate and install a new ruleset. In our experimental setup, this does not impact the evaluation, as $\Delta_t \gg \tau$ for both our baseline NIRS. However, more complex NIRS may need to account for it by extending the time range of r_t to $[t, t + \Delta_t + \tau]$, but using only $[t, t + \Delta_t]$ to update $\bar{X}_A$ and $\bar{X}_N$.

Metrics The impact of NIRS ruleset on network connections can be divided into four categories, mirroring the conventional classification of NIDS outputs into true positives, true negatives, false positives, and false negatives. Specifically, given a NIRS actions on a portion of network traffic, we can count the number of connections that are: correctly accepted (CA), wrongly accepted (WA), correctly blocked (CB), and wrongly blocked (WB). These values can be used to calculate NIRS performance metrics: the **correct block rate (CBR)**, i.e., the proportion

Algorithm 1 NIRS evaluation on a dataset $\mathbb{D}$ with constant Δt

Require: Dataset $\mathbb{D} = \{(x_i, y_i)\}_{i=1}^n$ of NetFlows and ground-truth labels, NIDS $\mathcal{D}$, NIRS $\mathcal{R}$, update interval Δt .

- 1: Initialize empty sets $\bar{X}_A \leftarrow \emptyset, \bar{X}_N \leftarrow \emptyset$
- 2: $t_{\min} \leftarrow \min_{i=1, \dots, n} t_i, t_{\max} \leftarrow \max_{i=1, \dots, n} t_i$
- 3: Initialize time $t \leftarrow t_{\min}$
- 4: Set initial ruleset r_t $\triangleright$ E.g., initialize r_t to always output 0 (accept all traffic)
- 5: Initialize $CB \leftarrow 0, WB \leftarrow 0, CA \leftarrow 0, WA \leftarrow 0$
- 6: **while** $t \leq t_{\max}$ **do**
- 7: **for each** x_i **where** $t \leq t_i \leq t + \Delta t$ **do**
- 8: $b_i \leftarrow r_t(x_i)$ $\triangleright$ Check if x_i is blocked by current ruleset
- 9: **if** $b_i = 1$ **then**
- 10: **if** $y_i = 1$ **then** $CB \leftarrow CB + 1$ $\triangleright$ Compare with ground truth
- 11: **else** $WB \leftarrow WB + 1$
- 12: **else**
- 13: **if** $y_i = 1$ **then** $WA \leftarrow WA + 1$
- 14: **else** $CA \leftarrow CA + 1$
- 15: $\hat{y}_i \leftarrow \mathcal{D}(x_i)$
- 16: **if** $\hat{y}_i = 1$ **then** $\bar{X}_A \leftarrow \bar{X}_A \cup \{x_i\}$ $\triangleright$ Update $\bar{X}_A, \bar{X}_N$ based on the NIDS output
- 17: **else** $\bar{X}_N \leftarrow \bar{X}_N \cup \{x_i\}$
- 18: $r_{t+\Delta t} \leftarrow \mathcal{R}(r_t, \bar{X}_A, \bar{X}_N)$ $\triangleright$ Update ruleset
- 19: $t \leftarrow t + \Delta t$ $\triangleright$ Update current time
- 20: **return** CB, WB, CA, WA

of malicious traffic that is correctly blocked

$$CBR = \frac{CB}{CB + WA} \quad (4)$$

and the **wrong block rate (WBR)**, i.e., the proportion of benign traffic that is incorrectly blocked

$$WBR = \frac{WB}{WB + CA} \quad (5)$$

These metrics are analogous to TPR and FPR, respectively, for NIDS. Both CBR and WBR can be computed either per-packet or per-NetFlow.

4 A demonstration of NIRS analysis

In this section we illustrate our proposed methodology by applying it to two examples of NIRS, denoted with $\mathcal{R}_1$ and $\mathcal{R}_2$, respectively. These two NIRS rely on the same setup and workflow, but use different rule-update functions. $\mathcal{R}_1$ is heuristic-based, while $\mathcal{R}_2$ leverages an LLM.

4.1 Two examples of NIRS

Both NIRS examples are based on a common setup in which the outputs of a NIDS (i.e., network connections labeled as either normal or alerts) are stored

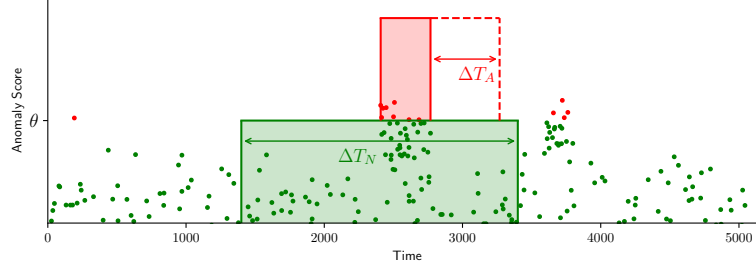


Fig. 2. Window system used to select samples from $\bar{X}_A$ and $\bar{X}_N$ in our baseline examples.

by two separated windows as shown in fig. 2. Normal traffic is collected using a sliding window $\bar{X}_N$ of fixed duration ΔT_N , meaning only the connections observed in the last ΔT_N seconds are used. Alerts, instead, are grouped using an incremental window $\bar{X}_A$ with a timeout of ΔT_A . The alert window continues to expand as long as new alerts arrive within ΔT_A seconds of the previous one; each new alert resets the timeout. Once the timeout is exceeded without new alerts, the window is flushed, under the assumption that alerts separated by more than ΔT_A seconds likely originate from unrelated incidents. The NIRS $\mathcal{R}(r_t, \bar{X}_A, \bar{X}_N)$ processes the contents of these two windows ($\bar{X}_A, \bar{X}_N$) along with the current firewall ruleset r_t and produces an updated ruleset r_{t+1} .

The rulesets r_t produced by both our baseline NIRS are collections of `iptables` `FORWARD` rules. At each update, they updated rulesets include exactly one new rule, except for cases where the NIRS generates an invalid rule³. To prevent uncontrolled growth of the ruleset, a cap of n_{rules} rules is enforced; when this threshold is exceeded, the oldest rule is discarded. The `iptables` rules should be enforced on Linux machines that are used as firewalls. Our assumptions regarding the deployment of these firewalls are specified in §4.2. The effect of these rules on the dataset’s NetFlows is simulated using a custom `iptables` parser, detailed in §7.1. To reduce the complexity of rule generation and parsing, we constrain the pool of allowable rules to a limited set of predefined formats, summarized in Table 2. The NIRS updates happen at a constant rate, every Δt seconds. Although this setup is characterized by several hyperparameters, an extensive tuning of these is outside the scope of this paper. In our evaluation, we simply fix these hyperparameters to arbitrary values, which are summarized in Table 3. It is worth noting that, while our implementations of NIRS rely on `iptables`, other types of firewalls or IPS can be employed to generate rulesets. For example, eBPF filters in Suricata⁴ could also be used.

³ This applies only to $\mathcal{R}_2$, since LLMs may produce invalid rules.

⁴ <https://docs.suricata.io/en/latest/capture-hardware/ebpf-xdp.html>

Description	Rule format
Block source IP or subnet	-A FORWARD -s <src_ip>[/<subnet>] -j DROP
Block destination IP or subnet	-A FORWARD -d <dst_ip>[/<subnet>] -j DROP
Block specific protocol for a destination IP/- subnet	-A FORWARD -d <dst_ip>[/<subnet>] -p <protocol> -j DROP
Block specific protocol's port for a destination IP/subnet	-A FORWARD -d <dst_ip>[/<subnet>] -p <protocol> -dport <dst_port> -j DROP

Table 2. Allowed `iptables` rules formats.

Heuristic-based NIRS ($\mathcal{R}_1$). This NIRS updates its ruleset through a simple heuristic process that evaluates both alerts and normal traffic. At each update, the NIRS produces a new rule of the form

-A FORWARD -s {{src_ip}} -j DROP

The `src_ip` is selected as the most frequent IP in $\bar{X}_A$. However, to mitigate the impact on benign traffic, a limit of $\epsilon = 0.1$ is set for the occurrence of `src_ip` in the current window of $\bar{X}_N$. If that limit is exceeded for the most frequent IP in the alert window, the second most frequent is selected, and so on.

LLM-based NIRS ($\mathcal{R}_2$). For this NIRS, we use an LLM agent that has access to $\bar{X}_A$ and $\bar{X}_N$, and is prompted to produce one `iptables` rule at every call. A very simple prompt (reported in §7.2) with few-shot examples was used. Moreover, to limit the LLM’s context window, only the last $k_A = 10$ alerts and $k_N = 10$ benign NetFlows are added to the prompt. The produced rules are applied only if they match one of the allowed formats in Table 2. After preliminary testing, we selected `llama3:8b` as our baseline model due to its reliable formatting and overall consistency. Reasoning models such as `deepseek-r1:8b` and `qwen3:8b` often exhibited undesirable behavior, including excessive generation loops during rule synthesis. While `qwen2.5:7b` was able to produce syntactically valid `iptables` rules, it frequently failed to follow the required encapsulation format, substituting quotes in place of the specified tags. `llama3:8b` managed to generate correctly formatted rules with reasonable latency without fine-tuning. Additionally, we confirmed that the model has inherent knowledge of `iptables`, as we were able to generate valid rules without providing examples. The average rule generation time (including text generation and rule parsing) was ≈ 1.6 seconds on an Nvidia GTX 2080 Ti GPU.

4.2 Dataset selection

To evaluate our baseline NIRS implementations, we selected a dataset that closely mirrors our intended deployment setup. Specifically, we prioritized datasets

Hyperparameter	Value
Normal-traffic window size ΔT_N	10min
Alert window timeout ΔT_A	1min
Interval between NIRS updates Δt	30min
Max number of rules per ruleset n_{rules}	10
$\mathcal{R}_1$ ’s benign block limit ϵ	0.01
$\mathcal{R}_2$ ’s # of input alerts k_A	10
$\mathcal{R}_2$ ’s # of input benign NetFlows k_N	10

Table 3. Hyperparameters used in our evaluation of the baseline NIRS.

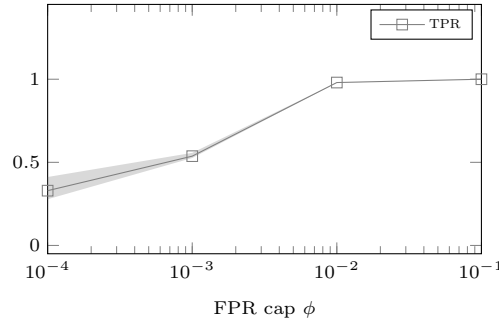


Fig. 3. TPR versus ϕ .

where NetFlows are collected at the router level, as this greatly facilitates matching them against the generated iptables rules. Several commonly used NIDS datasets (such as CIC-IDS 2017 [23], EdgeIoTSet [9], and TON-IoT [17]) were deemed unsuitable for our use case. In CIC-IDS 2017, traffic is captured at the host level within a private subnet behind a NAT, and external attacks are recorded as originating from the router’s IP address rather than the true source. While it is trivial to map internal private IPs to public ones, identifying the correct external attacker IPs that should replace the router IP is non-trivial. Similar issues are present in EdgeIoTSet, where traffic is also collected individually at the host level. In the case of TON-IoT, all hosts are connected through a vSwitch within the same subnet, making the application of `iptables` for routing-based mitigation infeasible. Instead, we opted to use the NB15 dataset [18], which is also widely used in the NIDS literature. In NB15, traffic is captured by a router, monitoring interactions across three main subnets: subnet 1 (59.166.0.0/24) and subnet 3 (149.171.126.0/24) generate normal traffic, while attacks originate from subnet 2 (175.45.176.0/24). These subnets exchange traffic among themselves and with other hosts through two routers, (linking subnets 10.40.85.0/24 and 10.40.182.0/24), one of which is used to capture traffic. Our experimental evaluation assumes router 1 is a Linux router running an instance of either $\mathcal{R}_1$ or $\mathcal{R}_2$ to dynamically update the `iptables` FORWARD table.

4.3 Experimental results

Ideal NIDS $\mathcal{D}^$* Our first experiment aimed to assess the effectiveness of our baseline NIRS implementations under ideal conditions. Specifically, we executed Algorithm 1 using a perfect NIDS $\mathcal{D}^*$ that labels all traffic correctly with no false positives or false negatives. This allows us to isolate and evaluate the rule-generation and enforcement capabilities of the NIRS without interference from detection errors. Under these assumptions, both $\mathcal{R}_1$ and $\mathcal{R}_2$ exhibited strong performance in terms of CBR and WBR, successfully mitigating the majority of malicious traffic. The heuristic-based $\mathcal{R}_1$ generated rules that blocked the IP addresses of each attacker from the subnet 175.45.176.0/24. However, it also in-

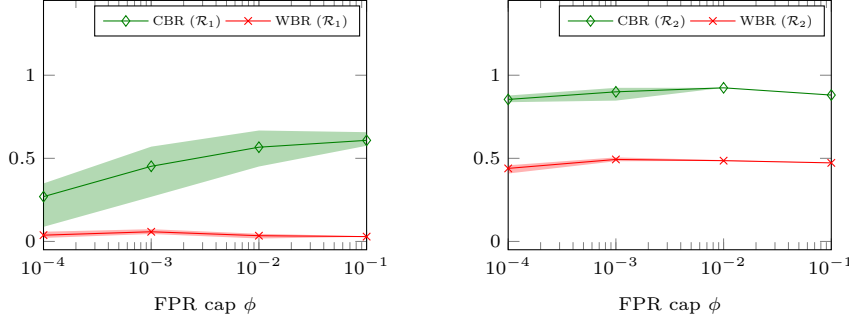


Fig. 4. Correct block rate (CBR) and wrong block rate (WBR) versus FPR cap ϕ for $\mathcal{R}_1$ (left) and $\mathcal{R}_2$ (right).

correctly blocked the IP address 149.171.126.12, which was a legitimate host that happened to be under attack and not generating benign traffic at the time of rule creation. In contrast, $\mathcal{R}_2$ immediately blocked the entire 175.45.176.0/24 subnet. This strategy yielded a high CBR of 97.55% while maintaining a relatively low WBR of 1.67%.

ML-based $\mathcal{D}_\phi$ To simulate a more realistic scenario, we replaced the ideal NIDS with an ML-based NIDS $\mathcal{D}_\phi$. We trained a Random Forest classifier (with 100 trees and max. depth 10) on the first 30% of the NetFlows, which were then excluded when calculating TPR, CBR, and WBR. Instead of using the Random Forest predicted labels directly, we extracted a probability value p_i for each NetFlow x_i derived from its Gini impurity. These probabilities can be interpreted as “anomaly scores” for the NetFlows. A threshold θ was then applied to the probabilities to enforce specific FPR caps $\phi = 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$, allowing us to examine how NIRS behavior changes under different volumes of false positives. The experiments were repeated over 5 trials, each initialized with a PRNG seed corresponding to the trial number to ensure reproducibility. As shown in Fig. 3, the true positive rate (TPR) of the NIDS increases as ϕ increases, following the expected trade-off curve. Due to the use of a fixed temporal split, the variability in TPR across trials remains minimal; however, the actual NetFlows labeled as benign or malicious vary, which has downstream effects on the NIRS performance.

These variations notably impact $\mathcal{R}_1$, which constructs rules by blocking individual IP addresses. Since $\mathcal{R}_1$ operates under a constraint to block less than 1% of the normal window $\bar{X}_N$, false negatives can significantly reduce its CBR. As illustrated in Fig. 4, $\mathcal{R}_1$ ’s CBR remained below 60% across all trials. On the upside, this conservative approach consistently maintained a low WBR, staying under 8%. This makes $\mathcal{R}_1$ more compatible with real-world deployment, as long as its rule generation is supervised or reviewed by a human operator. In contrast, $\mathcal{R}_2$ prioritized broader mitigation actions, often generating rules that blocked entire subnets. While this strategy enabled a high CBR (ranging between 85% and

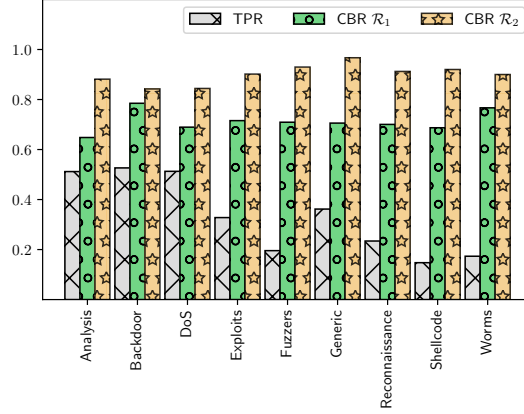


Fig. 5. CBR by attack for $\phi = 0.1$.

91%), it also disrupted around 45% of benign NetFlows. Attempts to reduce this over-blocking (through negative prompting, e.g., explicitly instructing the model not to block traffic from 59.166.0.0/24; or by warning it about the risk of false positives) were ineffective. Moreover, $\mathcal{R}_2$ exhibited hallucination behavior, claiming in multiple instances to block a single IP while actually generating rules that blocked whole subnets. This bias toward subnet-level blocking may be attributed to the examples of `iptables` rules fed to `llama3:8b` during training. Future work involving fine-tuning could help mitigate both the hallucinations and the unresponsiveness to negative prompts.

Figure 5 compares the TPR and the resulting CBRs for $\mathcal{R}_1$ and $\mathcal{R}_2$ for $\phi = 0.1$. Notably, even for low TPR values, the NIRS were often still able to block most of the malicious flows, as long as a sufficient number of alerts were generated. However, this effectiveness is also caused by the attack originating from a small number of IP addresses. Further experimentation is required to understand how the NIRS perform against distributed attacks, where malicious activity may be spread across a wide range of sources [10], potentially diminishing the CBR.

Lessons learned and future directions Although our analysis was limited to a single dataset, the primary objective was to demonstrate how NIRS systems can be systematically assessed using publicly available data, rather than drawing definitive conclusions. Nevertheless, several meaningful insights emerged from our analysis can help shape more refined approaches. The two baseline NIRS we explored, though clearly not suitable for production environments, highlight different strengths and weaknesses. The heuristic-based system ($\mathcal{R}_1$) revealed that simple safeguards like a threshold based on normal traffic help minimizing disruptions, even when the FPR is high. On the other hand, NIRS based on lightweight out-of-the-box LLMs like `llama3:8b` confirm their ability to generate

firewall rules directly from alert data. However, they currently struggle with following specific instructions, especially negative prompts. This is a known issue with LLMs, and recent works have highlighted the difficulty of developing a general prompt template for network-related tasks [7]. Fine-tuning the model or guiding its reasoning process may help mitigate these issues [27].

5 Discussion

Limitations of our proposed evaluation methodology While metrics such as the CBR and WBR provide a quantitative measure of a NIRS’s performance, they are not sufficient on their own to determine whether an attack has been effectively mitigated. In the case of DoS, a high rate of blocked connections typically correlates with reduced service disruption, offering a relatively clear indication of successful mitigation. However, this relationship does not hold across all attack types, and more nuanced evaluation protocols are needed when focusing on specific attack vectors. Still, CBR and WBR remain valuable as general-purpose metrics, allowing for broad comparisons across different families of attacks. Another aspect to be considered is related to NIDS datasets. While dataset-based evaluation is probably one of the strongest advantages of NIRS, it still presents limitations. Datasets do not account for adaptive adversaries who deliberately attempt to evade detection and response mechanisms. Modeling such adversaries is complex, especially within the constraints of pre-collected data. This limitation is not unique to NIRS, but rather affects most data-driven approaches to cybersecurity.

Including NIRS in more complex solutions NIRS are not intended to function as standalone components, much like how NIDS should not be the only systems in place to monitor the network. In network security, it is widely recognized that no single solution can address all scenarios; instead, components must be integrated into broader, more complex architectures tailored to the specific needs of the system [24]. For instance, in large enterprise environments where the volume of alerts can overwhelm security analysts, NIRS could serve as alert aggregation tools that distill raw NIDS alerts into actionable summaries and predefined response strategies. In such deployments, the output of a NIRS might be redirected into a Security Information and Event Management (SIEM) platform or a log search engine like Elasticsearch. Whitelisting mechanisms could also be integrated to avoid blocking critical connections.

Resilience to adversarial attacks As NIRS decisions rely heavily on the accuracy of NIDS predictions, their effectiveness may be undermined by adversarial attacks specifically designed to evade NIDS analysis. Moreover, because our proposed notion of NIRS is primarily intended for LLM-based applications – particularly those using RAG or AI agents as decision makers – there are legitimate concerns about potential exploitation through techniques such as prompt injection [16]. This risk, however, can be effectively mitigated by adopting a

predefined prompt template in which the only external inputs are numerical indicators or other objective information (e.g., IP addresses or protocols), as was implemented for $\mathcal{R}_2$ in this paper.

6 Conclusion

In this paper, we motivated and introduced the definition of NIRS, providing a practical methodology that can be used to evaluate these systems. We also showed our evaluation methodology at work, applying it to two baseline NIRS solution. Future research may use our work as a starting point to build more sophisticated NIRS solutions. For example, more complex NIRS may use additional NetFlow information and metadata, such as domain names and host roles. Additionally, LLM-based solutions can be improved via fine-tuning or guided reasoning to encourage model compliance with negated prompting.

Acknowledgments. This work has been partially supported by the French National Research Agency under the France 2030 label (Superviz ANR-22-PECY-0008). The views reflected herein do not necessarily reflect the opinion of the French government.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Antonioli, D., Tippenhauer, N.O.: Minicps: A toolkit for security research on cps networks. In: Proceedings of the First ACM workshop on cyber-physical systems-security and/or privacy. pp. 91–100 (2015)
2. Apruzzese, G., Laskov, P., Schneider, J.: Sok: Pragmatic assessment of machine learning for network intrusion detection. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P). pp. 592–614. IEEE (2023)
3. Apruzzese, G., Pajola, L., Conti, M.: The cross-evaluation of machine learning-based network intrusion detection systems. *IEEE Transactions on Network and Service Management* **19**(4), 5152–5169 (2022)
4. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don’ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
5. Bashendy, M., Tantawy, A., Erradi, A.: Intrusion response systems for cyber-physical systems: A comprehensive survey. *Computers & Security* **124**, 102984 (2023)
6. Booij, T.M., Chiscop, I., Meeuwissen, E., Moustafa, N., Den Hartog, F.T.: Ton_iiot: The role of heterogeneity and the need for standardization of features and attack types in iiot network intrusion data sets. *IEEE Internet of Things Journal* **9**(1), 485–496 (2021)
7. Donadel, D., Marchiori, F., Pajola, L., Conti, M.: Can llms understand computer networks? towards a virtual system administrator. In: 2024 IEEE 49th Conference on Local Computer Networks (LCN). pp. 1–10. IEEE (2024)

8. Etchezarreta, X., Garitano, I., Iturbe, M., Zurutuza, U.: Software-defined networking approaches for intrusion response in industrial control systems: A survey. *International Journal of Critical Infrastructure Protection* **42**, 100615 (2023)
9. Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H.: Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *IEEE Access* **10**, 40281–40306 (2022)
10. Friji, H., Mavromatis, I., Sanchez-Mompo, A., Carnelli, P., Olivereau, A., Khan, A.: Multi-stage attack detection and prediction using graph neural networks: An iot feasibility study. In: 2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). pp. 620–627. IEEE (2023)
11. Hughes, K., McLaughlin, K., Sezer, S.: A model-free approach to intrusion response systems. *Journal of Information Security and Applications* **66**, 103150 (2022)
12. Iannucci, S., Huang, Y.C., Bertino, E.: A performance evaluation of deep reinforcement learning for model-based intrusion response. In: 2019 IEEE 4th International Workshops on Foundations and Applications of Self* Systems (FAS* W). pp. 83–88. IEEE (2019)
13. Kshetri, N.: Economics of artificial intelligence in cybersecurity. *IEEE IT Professional* **23**(2), 7–11 (2021). <https://doi.org/10.1109/MITP.2021.3059231>
14. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the cids2017 dataset and the significant differences in detection performances it makes. In: International Conference on Risks and Security of Internet and Systems. pp. 18–33. Springer (2022)
15. Law, Y.W., Alpcan, T., Palaniswami, M.: Security games for risk minimization in automatic generation control. *IEEE Transactions on Power Systems* **30**(1), 223–232 (2014)
16. Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., et al.: Prompt injection attack against llm-integrated applications. arXiv preprint arXiv:2306.05499 (2023)
17. Moustafa, N.: A new distributed architecture for evaluating ai-based security systems at the edge: Network ton_iot datasets. *Sustainable Cities and Society* **72**, 102994 (2021)
18. Moustafa, N., Slay, J.: Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In: 2015 military communications and information systems conference (MilCIS). pp. 1–6. IEEE (2015)
19. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L.: {TESSERACT}: Eliminating experimental bias in malware classification across space and time. In: 28th USENIX security symposium (USENIX Security 19). pp. 729–746 (2019)
20. Phan, T.V., Bauschert, T.: Deepair: Deep reinforcement learning for adaptive intrusion response in software-defined networks. *IEEE Transactions on Network and Service Management* **19**(3), 2207–2218 (2022)
21. Piedrahita, A.F.M., Gaur, V., Giraldo, J., Cardenas, A.A., Rueda, S.J.: Virtual incident response functions in control systems. *Computer Networks* **135**, 147–159 (2018)
22. Rose, J.R., Swann, M., Grammatikakis, K.P., Koufos, I., Bendiab, G., Shiaeles, S., Kolokotronis, N.: Ideres: Intrusion detection and response system using machine learning and attack graphs. *Journal of Systems Architecture* **131**, 102722 (2022)
23. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., et al.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* **1**(2018), 108–116 (2018)

24. Sheeraz, M., Paracha, M.A., Haque, M.U., Durad, M.H., Mohsin, S.M., Band, S.S., Mosavi, A.: Effective security monitoring using efficient siem architecture. *Hum.-Centric Comput. Inf. Sci* **13**, 1–18 (2023)
25. Team., M.D.R.: Cyberbattlesim. <https://github.com/microsoft/cyberbattlesim> (2021), created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei.
26. Yarygina, T., Otterstad, C.: A game of microservices: Automated intrusion response. In: Distributed Applications and Interoperable Systems: 18th IFIP WG 6.1 International Conference, DAIS 2018, Held as Part of the 13th International Federated Conference on Distributed Computing Techniques, DisCoTec 2018, Madrid, Spain, June 18–21, 2018, Proceedings 18. pp. 169–177. Springer (2018)
27. Zhang, Z., Yao, Y., Zhang, A., Tang, X., Ma, X., He, Z., Wang, Y., Gerstein, M., Wang, R., Liu, G., et al.: Igniting language intelligence: The hitchhiker’s guide from chain-of-thought reasoning to language agents. *ACM Computing Surveys* **57**(8), 1–39 (2025)
28. Zonouz, S.A., Khurana, H., Sanders, W.H., Yardley, R.: Rre: A game-theoretic intrusion response and recovery engine. *IEEE Transactions on Parallel and Distributed Systems* **25**(2), 395–406 (2013)

7 Appendix

7.1 Rule parser

To match `iptables` rules with NetFlows, we used a custom parser that first converts the rule in a JSON representation. For example, the `iptables` rule `-A FORWARD -d 10.122.2.10 -p tcp -dport 22 -j DROP` is mapped to

```
{
  "option": "-A",
  "table": "FORWARD",
  "src_ip": "any",
  "dst_ip": "10.122.2.10",
  "protocol": "tcp",
  "src_port": "any",
  "dst_port": "22",
  "jump": "DROP",
}
```

The JSON rule is then matched against the NetFlows’ fields. Since `iptables` apply to packets, NetFlows are matched in case packets coming from either directions are matched, as that implies that the connection was disrupted. In other words, the rule above also matches NetFlows with `src_ip` 192.14.2.10, `protocol` TCP, `src_port` 22, as long as the number of source-to-destination bytes in the original NetFlow is > 0 . Subnets are parsed as regular IP addresses, but a different matching function is used, which compares only prefixes.

7.2 LLM-based NIRS details

Prompts To generate `iptables` rules with LLMs, we used the system and user prompts shown in fig. 6.

When no rules is in place, the `iptables` status is set to `[Empty]`. Otherwise, it displays the current list of rules.

$\mathcal{R}_2$ prompts

System prompt: You are a network security engineer. You are tasked with monitoring incoming malicious and benign traffic, and writing one iptables rule accordingly. You will observe examples of benign flows and malicious flows. You will also have access to the current iptables status. Based on this information, you will write one single iptables rule, which should be enclosed within `<rule></rule>` tags. Output only one iptables DROP rule to append to the FORWARD table. The rule must block most of the malicious flows and must not block most of the benign flows.

Valid formats for the rule include:

{{accepted_formats}}

DO NOT USE RULES WITH DIFFERENT FORMATS. The `/<subnet>` is optional. Examples of valid rules:

{{few_shot_examples}}

User prompt: Malicious flows:

{{malicious_flows}}

Benign flows:

{{benign_flows}}

Iptables status:

{{iptables_status}}

Accepted formats:

```
-A FORWARD -s <src_ip>/<subnet> -j DROP
-A FORWARD -d <dst_ip>/<subnet> -j DROP
-A FORWARD -d <dst_ip>/<subnet> -p <protocol> -j DROP
```

Few-shot examples:

```
-A FORWARD -s 10.25.0.41 -j DROP
-A FORWARD -s 172.21.0.1/24 -j DROP
-A FORWARD -d 208.42.13.2 -j DROP
-A FORWARD -d 113.0.201.5 -p icmp
-A FORWARD -d 32.153.41.11 -p tcp --dport 22
```

Fig. 6. System and user prompt formats used for $\mathcal{R}_2$.

Example of LLM output An example of output generated by llama3:8b is as follows:

```
<rule>-A FORWARD -s 175.45.176.0/24 -j DROP</rule>

This rule blocks all traffic from the malicious IP range
(175.45.176.0/24) to any destination, effectively blocking
most of the malicious flows while allowing benign traffic to pass through.
```

Example of LLM hallucinations In some cases, the explanation provided by the LLM did not correctly describe the effect of the rule on the network traffic. For example:

```
<rule>-A FORWARD -s 59.166.0.3/24 -j DROP</rule>

This rule blocks traffic from IP address 59.166.0.3, which is
the source IP address for all malicious flows.
This should block most of the malicious traffic while allowing
benign traffic to pass through.
```

While the the output claims that this rule only blocks 59.166.0.3, `iptables` prioritizes netmasks, meaning that this rule blocks any IP in the 59.166.0.0/24 subnet.